ResearchLens: n8n Workflow Integration

Implementation notes, configuration, testing, troubleshooting, and current limitations

Prepared 12 October 2026

1. Overview

ResearchLens is an AI-assisted web research application. Its backend is built with Python and FastAPI, and it can retrieve webpage content, send extracted text for summarization through Google Gemini, and return a report that can be exported as HTML. The n8n integration automates calling the deployed ResearchLens API, converting the response to an HTML file, and saving that file locally.

The purpose of this integration was to demonstrate practical workflow automation: scheduling a task, calling a REST API, transforming the result into a file, and writing the file to disk. The workflow was built and tested locally on a Mac using n8n started from the terminal.

Deployed API base: https://research-lens-ai-research.vercel.app/

2. Workflow architecture

The workflow contains four connected nodes in this order:

- Schedule Trigger → starts the workflow at a configured time.
- HTTP Request → sends a POST request to the ResearchLens HTML export endpoint.
- Convert to File → converts the returned report content into a text/HTML file.
- Read/Write Files from Disk → writes the resulting file to a local directory.

The screenshot below shows all four nodes connected and green check marks from a successful execution. The green checks document a successful run; they do not guarantee every future run will succeed.

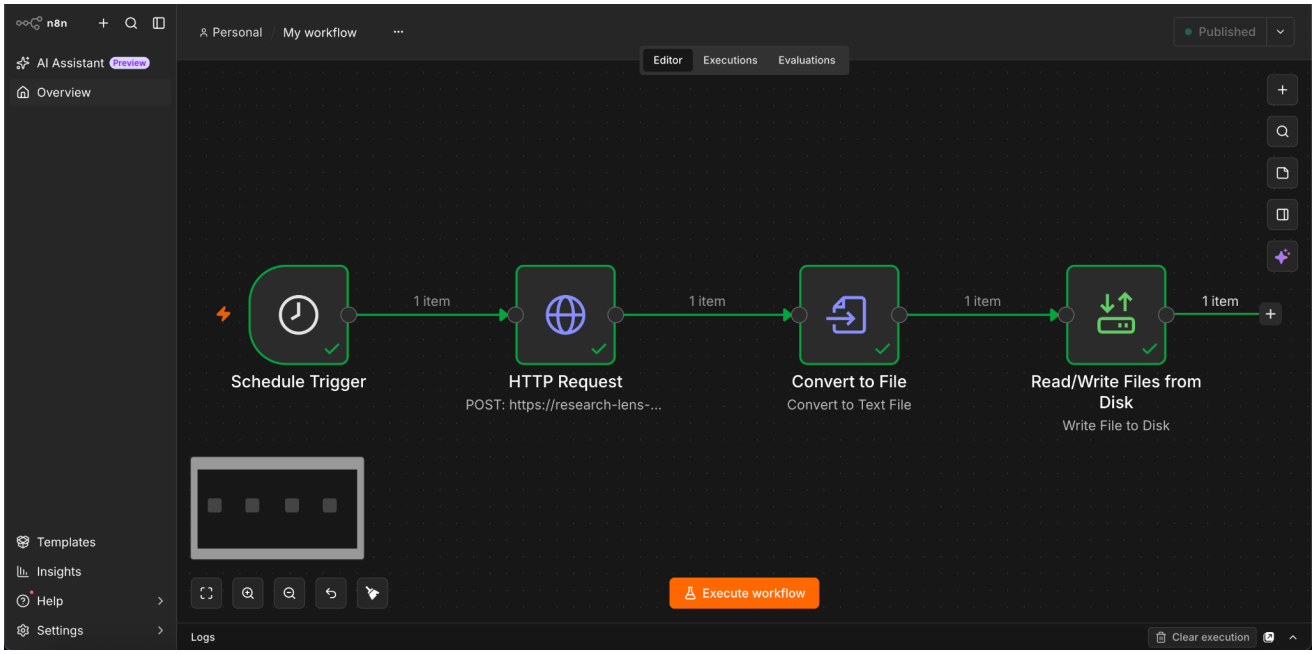


Figure 1. n8n workflow showing the four connected nodes and a successful test execution.

3. Starting n8n locally

n8n was run locally from a terminal using npx n8n. This starts the n8n service for the current local environment and makes its workflow editor available in a browser.

Practical notes:

- Keep the terminal process running while expecting scheduled workflows to execute.
- Keep the Mac awake and connected to the network at the scheduled time.
- Closing the browser tab does not necessarily stop n8n, but stopping the terminal process or shutting down the computer prevents the local scheduler from running.
- This is a local development setup, not a 24/7 hosted automation service.

4. Node-by-node configuration

4.1 Schedule Trigger

The Schedule Trigger is the workflow entry point. The intended recurring schedule was daily at 7:00 AM, with the workflow timezone set to Asia/Kolkata. During testing, the schedule was temporarily changed to other times (including 1:00 AM and 11:00 AM) to verify execution. Before relying on the workflow, confirm the final schedule in the node and publish the saved workflow.

4.2 HTTP Request

The HTTP Request node sends a POST request to the deployed HTML export endpoint:

```
https://research-lens-ai-research.vercel.app/research/export/html
```

The request body is JSON. The tested body used the TechCrunch homepage as the source URL and provided instructions to summarize technology industry news, company announcements, product launches, startup funding, and emerging trends. It requested headlines and dates where available, explanations of why each development matters, and no invented facts.

```
{\n "url": "https://techcrunch.com/",\n "focus": "Summarize the latest technology industry news, major company announcements, product launches, startup funding, and emerging trends. Include headlines and dates when available. Explain why each development matters and its potential industry impact. Use only information supported by the webpage; do not invent facts.",\n "demo": false\n}
```

The demo flag was set to false so the request would use the live Gemini-backed path rather than a demonstration response. The request used no custom authentication header in the node configuration shown during setup. Do not commit API keys or private credentials to the workflow export or repository.

4.3 Convert to File

The Convert to File node was configured with the operation Convert to Text File. It takes the HTTP Request response and prepares the report content as a file for the next node. The resulting file content is HTML, so it can be opened in a browser.

4.4 Read/Write Files from Disk

The final node was configured with the operation Write File to Disk. The output path used during testing was:

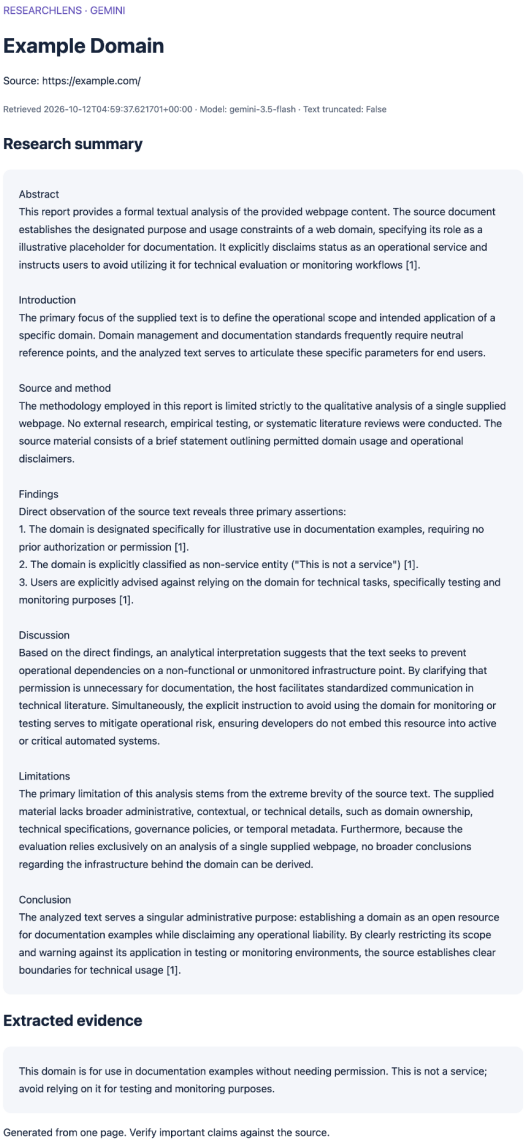
```
~/.n8n-files/ResearchLens/reports/researchlens-report.html
```

This path points to a local directory in the user's home folder. The filename was configured as a fixed name, so subsequent successful runs overwrite the previous report. A dated filename pattern could be introduced later if keeping a history of reports is required.

5. Successful output

The HTML screenshot shows a report page with a title, source URL, retrieval/model metadata, research summary, extracted evidence, and a note reminding readers to verify important claims against the source. The screenshot provided is a format/output test using https://example.com/ and metadata showing gemini-3.5-flash at the time it was generated. It demonstrates that the HTML export pipeline produced a readable report; it is not itself evidence that the later TechCrunch daily-news prompt was completed successfully.

Earlier tests also successfully ran the workflow and generated local report files. The workflow screenshot and report screenshot should be kept together in the project documentation: one shows the automation, and the other shows its output.


RESEARCHLENS · GEMINI

Example Domain

Source: https://example.com/

Retrieved 2026-10-12T04:59:37.621701+00:00 · Model: gemini-3.5-flash · Text truncated: False

Research summary

Abstract
This report provides a formal textual analysis of the provided webpage content. The source document establishes the designated purpose and usage constraints of a web domain, specifying its role as a illustrative placeholder for documentation. It explicitly disclaims status as an operational service and instructs users to avoid utilizing it for technical evaluation or monitoring workflows [1].

Introduction
The primary focus of the supplied text is to define the operational scope and intended application of a specific domain. Domain management and documentation standards frequently require neutral reference points, and the analyzed text serves to articulate these specific parameters for end users.

Source and method
The methodology employed in this report is limited strictly to the qualitative analysis of a single supplied webpage. No external research, empirical testing, or systematic literature reviews were conducted. The source material consists of a brief statement outlining permitted domain usage and operational disclaimers.

Findings
Direct observation of the source text reveals three primary assertions:
1. The domain is designated specifically for illustrative use in documentation examples, requiring no prior authorization or permission [1].
2. The domain is explicitly classified as non-service entity ("This is not a service") [1].
3. Users are explicitly advised against relying on the domain for technical tasks, specifically testing and monitoring purposes [1].

Discussion
Based on the direct findings, an analytical interpretation suggests that the text seeks to prevent operational dependencies on a non-functional or unmonitored infrastructure point. By clarifying that permission is unnecessary for documentation, the host facilitates standardized communication in technical literature. Simultaneously, the explicit instruction to avoid using the domain for monitoring or testing serves to mitigate operational risk, ensuring developers do not embed this resource into active or critical automated systems.

Limitations
The primary limitation of this analysis stems from the extreme brevity of the source text. The supplied material lacks broader administrative, contextual, or technical details, such as domain ownership, technical specifications, governance policies, or temporal metadata. Furthermore, because the evaluation relies exclusively on an analysis of a single supplied webpage, no broader conclusions regarding the infrastructure behind the domain can be derived.

Conclusion
The analyzed text serves a singular administrative purpose: establishing a domain as an open resource for documentation examples while disclaiming any operational liability. By clearly restricting its scope and warning against its application in testing or monitoring environments, the source establishes clear boundaries for technical usage [1].

Extracted evidence

This domain is for use in documentation examples without needing permission. This is not a service; avoid relying on it for testing and monitoring purposes.

Generated from one page. Verify important claims against the source.

Figure 2. Example generated HTML report (example.com test; shown to document output format).

6. Error history and troubleshooting

6.1 Gemini quota / rate-limit investigation

During troubleshooting, the Google AI Studio rate-limit dashboard showed usage for Gemini 3.6 Flash above the displayed daily request limit in one screenshot (24/20), while Gemini 3.8 Flash showed 2/20 requests and low token use. These values are time-specific dashboard observations and can change. The screenshots helped distinguish model-level usage from the current request failure.

The user initially configured gemini-3.5-flash. Google's model routing/deprecation behavior can route requests for older model IDs to a newer model; the observed dashboard usage for Gemini 3.6 Flash therefore did not prove that the current deployment was still configured for 3.5. The current Vercel request log explicitly identified gemini-3.8-flash.

6.2 Current 503 Service Unavailable

A later Vercel log entry for POST /research/export/html showed a 502 response from the deployed ResearchLens endpoint, with the nested upstream request to Gemini returning HTTP/1.1 503 Service Unavailable for gemini-3.8-flash:generateContent. The user-facing n8n error described Gemini as temporarily overloaded.

Interpretation: the application reached the Gemini API, but the upstream model call failed with a 503. This points to a service availability/overload problem at the time of the request; it is different from a confirmed invalid API key. The log alone does not prove the underlying root cause, so the diagnosis should remain provisional.

Actions already taken: GEMINI_MODEL was recreated in Vercel as a readable Config value set to gemini-3.8-flash for Production, and the application was redeployed. The subsequent log still showed a 503 from Gemini 3.8 Flash. The GEMINI_API_KEY was left in its existing Secret configuration.

6.3 Why older logs were not visible

The Vercel Logs view was filtered by a specific deployment ID and a Last 30 minutes timeline. Older entries can be hidden by those filters or fall outside the selected time window. Resetting the filters and widening the time range can help locate previous events. Redeployment does not necessarily mean older logs were deleted; the selected view may simply be scoped to a particular deployment or time range.

6.4 Recommended next improvements

- Add bounded retry logic with exponential backoff for transient 503/429 responses where appropriate.
- Return clear API error messages and preserve upstream status details in server logs without exposing secrets.
- Test the same model with a short prompt directly in Google AI Studio to help distinguish a model/service issue from an application integration issue.
- Consider a fallback model only after verifying model availability, access, and quota; do not rotate API keys or create new projects without evidence that the key/project is the problem.
- Use a dated output filename if reports should be archived rather than overwritten.

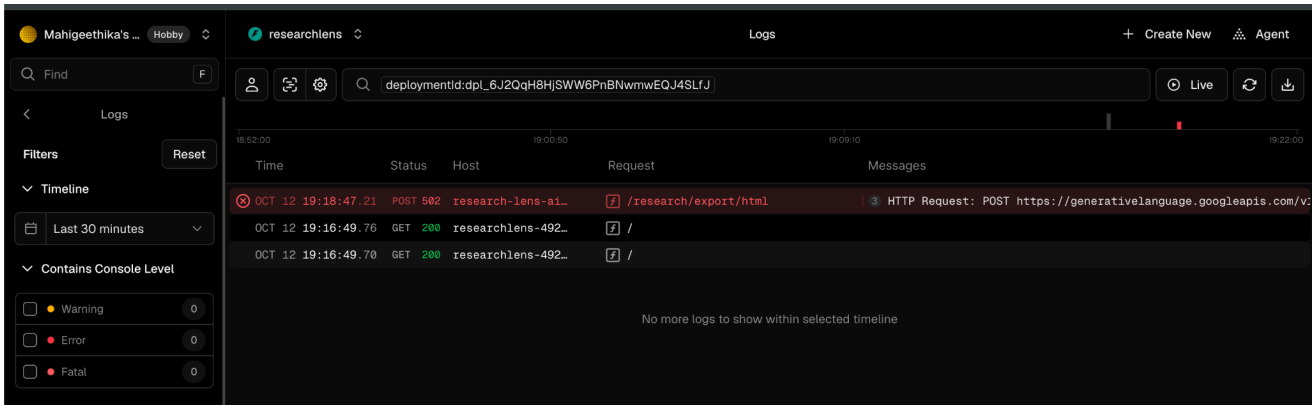


Figure 3. Vercel log showing 502 from the app and upstream 503 from Gemini 3.8 Flash.

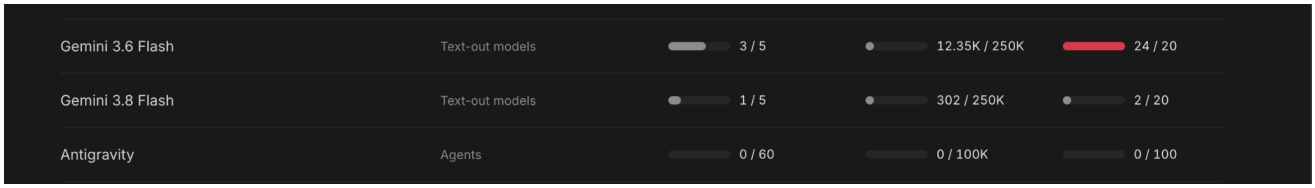


Figure 4. Google AI Studio rate-limit dashboard captured during troubleshooting; quota figures are time-specific.

7. Security and reproducibility

- Never include GEMINI_API_KEY or other credentials in screenshots, exported workflow JSON, README files, or Git commits.
- Before committing an n8n workflow export, inspect it for authorization headers, tokens, and sensitive local paths.
- Anyone importing the workflow needs to set their own endpoint and local output path and ensure their n8n process has permission to write to that folder.
- A local schedule depends on the machine and n8n process being available at run time. For unattended, always-on execution, a continuously running hosted/self-hosted n8n instance would be needed.

8. Current status

Status as of 12 October 2026: the n8n workflow has been assembled and successfully executed in earlier tests, and an HTML report has been generated locally. The latest recorded run failed because the Gemini upstream request returned 503 Service Unavailable, causing the deployed ResearchLens endpoint to return 502 to n8n. The workflow is therefore a valid implementation and integration demonstration, but unattended daily reliability is not yet confirmed.

Suggested portfolio wording: “Built an n8n workflow integrating a scheduled trigger, REST API request, file conversion, and local HTML report storage for an AI-powered research application. Tested end-to-end execution and investigated upstream Gemini API availability errors.”

9. Official reference

n8n documentation: https://docs.n8n.io/ — official documentation for workflow concepts, built-in nodes, scheduling, HTTP requests, file conversion, and file system operations.

Google Gemini API troubleshooting: https://ai.google.dev/gemini-api/docs/troubleshooting — consult the current documentation for API error meanings, quotas, and transient failure handling.